

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const fs = require('fs');
const Anthropic = require('@anthropic-ai/sdk');

const app = express();
app.use(cors());
app.use(express.json({ limit: '1mb' }));
app.use(express.static(path.join(__dirname, 'public')));

const PORT = process.env.PORT || 3000;
const API_KEY = process.env.ANTHROPIC_API_KEY;

if (!API_KEY) {
  console.warn('\n[KUJDES] ANTHROPIC_API_KEY nuk është vendosur në .env – kërkesa
}

const anthropic = new Anthropic({ apiKey: API_KEY || 'missing' });

const DATA_DIR = path.join(__dirname, 'data');
if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true });

function sessionFile(sessionId, kind) {
  const safe = String(sessionId || 'default').replace(/^[a-zA-Z0-9_-]/g, '').slice(1)
  return path.join(DATA_DIR, `${kind}_${safe}.json`);
}

function readJSON(file, fallback) {
  try { return JSON.parse(fs.readFileSync(file, 'utf8')); } catch (e) { return fa
}

function writeJSON(file, data) {
  fs.writeFileSync(file, JSON.stringify(data, null, 2));
}

function getSessionId(req) {
  return req.header('x-shoku-session') || 'default';
}

/* ----- MEMORY API ----- */
app.get('/api/memory', (req, res) => {
  res.json({ memory: readJSON(sessionFile(getSessionId(req), 'memory'), []) });
});

app.delete('/api/memory/:index', (req, res) => {
  const file = sessionFile(getSessionId(req), 'memory');

```

```

    const mem = readJSON(file, []);
    const idx = parseInt(req.params.index, 10);
    if (idx >= 0 && idx < mem.length) mem.splice(idx, 1);
    writeJSON(file, mem);
    res.json({ memory: mem });
  });
app.delete('/api/memory', (req, res) => {
  writeJSON(sessionFile(getSessionId(req), 'memory'), []);
  res.json({ memory: [] });
});

/* ----- TODOS API ----- */
app.get('/api/todos', (req, res) => {
  res.json({ todos: readJSON(sessionFile(getSessionId(req), 'todos'), []) });
});
app.post('/api/todos', (req, res) => {
  const file = sessionFile(getSessionId(req), 'todos');
  const todos = readJSON(file, []);
  const text = String(req.body.text || '').trim().slice(0, 300);
  if (!text) return res.status(400).json({ error: true, message: 'Teksti i detyrë' });
  todos.push({ id: Date.now().toString(36) + Math.random().toString(36).slice(2),
    text: text });
  writeJSON(file, todos);
  res.json({ todos });
});
app.patch('/api/todos/:id', (req, res) => {
  const file = sessionFile(getSessionId(req), 'todos');
  const todos = readJSON(file, []);
  const t = todos.find(t => t.id === req.params.id);
  if (t) t.done = !!req.body.done;
  writeJSON(file, todos);
  res.json({ todos });
});
app.delete('/api/todos/:id', (req, res) => {
  const file = sessionFile(getSessionId(req), 'todos');
  let todos = readJSON(file, []);
  todos = todos.filter(t => t.id !== req.params.id);
  writeJSON(file, todos);
  res.json({ todos });
});

/* ----- TOOL DEFINITIONS (real Anthropic tool-use) ----- */
const TOOLS = [
  {
    name: 'vendos_mjetin',
    description: 'Thirrë këtë çdo herë kur kërkesa e personit qartazi kërkon diçk',
    input_schema: {
      type: 'object',

```

```

    properties: {
      mjeti: {
        type: 'string',
        enum: ['kerkim_internet', 'muzike_ai', 'video_ai', 'organizim_detyrash'],
      },
      arsyeja: { type: 'string', description: 'Pse u zgjodh ky mjet, shkurt, në' },
      required: ['mjeti']
    }
  },
  {
    name: 'ruaj_kujtim',
    description: 'Ruaj një fakt të qëndrueshëm dhe me vlerë rreth personit ose pr
input_schema: {
  type: 'object',
  properties: { shenimi: { type: 'string', description: 'Përshkrim i shkurtër
    required: ['shenimi']
  }
},
  {
    name: 'menaxho_detyren',
    description: 'Shto, kryej, ose fshi një detyrë në listën e detyrave ditore të
input_schema: {
  type: 'object',
  properties: {
    aksioni: { type: 'string', enum: ['shto', 'fshij', 'kryer'] },
    teksti: { type: 'string', description: 'Teksti/emri i detyrës' }
  },
  required: ['aksioni']
}
}
];

```

```

function buildSystemPrompt(memory) {
  let mem = '';
  if (memory && memory.length) {
    mem = '\n\nÇka di tashmë për shokun tënd nga bisedat e mëparshme (përdore vet
  }
  return `Ti je SHOKU, një shok/mikeshë personale me AI, e krijume posaçërisht pë

```

Rregullat e tua:

- Përgjigju gjithmonë në shqip, në mënyrë të ngrohtë, njerëzore dhe të drejtpërdr
- Personi nuk di dhe s'ka nevojë të dijë "prompt-e" apo terma teknike. Kupto qëll
- Nëse ideja është e paqartë, bëj një supozim të arsyeshëm dhe vazhdo, në vend që
- Ji konciz kur mundesh – flet si shok në chat, jo si ese. Ndaji përgjigjet e gja
- Kur dikush sjell një ide, projekt, film, këngë, mësim – ndihmoje ta zhvillojë h

Mjetet që ke në dispozicion:

- organizim_detyrash: REALISHT AKTIV. Kur përdoruesi kërkon të shtosh/kryesh/fshi
- mesim, bisede_e_zakonshme: gjithmonë aktive, kjo është biseda kryesore – s'ke n
- kerkim_internet, muzike_ai, video_ai: STRUKTURA ekziston, por LIDHJA reale ende

Thirr ruaj_kujtim shumë rrallë – vetëm për fakte vërtet të qëndrueshme (emër proj
}

```
/* ----- CHAT (real model call + tool loop) ----- */
```

```
app.post('/api/chat', async (req, res) => {
```

```
  if (!API_KEY) {
```

```
    return res.status(500).json({
```

```
      error: true,
```

```
      message: 'ANTHROPIC_API_KEY mungon në server (.env). Shto çelësin në skedar
```

```
    });
```

```
  }
```

```
  const sessionId = getSessionId(req);
```

```
  const { messages } = req.body;
```

```
  if (!Array.isArray(messages) || messages.length === 0) {
```

```
    return res.status(400).json({ error: true, message: 'Kërkesa s\'ka mesazhe bi
```

```
  }
```

```
  const memory = readJSON(sessionFile(sessionId, 'memory'), []);
```

```
  const system = buildSystemPrompt(memory);
```

```
  let working = messages.map(m => ({ role: m.role, content: m.content }));
```

```
  let usedTool = null;
```

```
  try {
```

```
    for (let guard = 0; guard < 4; guard++) {
```

```
      const resp = await anthropic.messages.create({
```

```
        model: 'claude-sonnet-4-6',
```

```
        max_tokens: 1000,
```

```
        system,
```

```
        tools: TOOLS,
```

```
        messages: working
```

```
      });
```

```
      const toolUses = resp.content.filter(b => b.type === 'tool_use');
```

```
      const textOut = resp.content.filter(b => b.type === 'text').map(b => b.text
```

```
      if (toolUses.length === 0) {
```

```
        return res.json({ reply: textOut, tool: usedTool });
```

```
      }
```

```
      working.push({ role: 'assistant', content: resp.content });
```

```

const toolResults = [];
for (const tu of toolUses) {
  let resultText = 'ok';
  if (tu.name === 'vendos_mjetin') {
    usedTool = tu.input.mjeti;
    resultText = `Mjeti u regjistrua: ${tu.input.mjeti}`;
  } else if (tu.name === 'ruaj_kujtim') {
    const file = sessionFile(sessionId, 'memory');
    const mem = readJSON(file, []);
    if (tu.input.shenimi && !mem.includes(tu.input.shenimi)) {
      mem.push(tu.input.shenimi);
      writeJSON(file, mem);
    }
    resultText = 'U ruajt kujtimi.';
  } else if (tu.name === 'menaxho_detyren') {
    const file = sessionFile(sessionId, 'todos');
    let todos = readJSON(file, []);
    const needle = (tu.input.teksti || '').toLowerCase();
    if (tu.input.aksioni === 'shto' && tu.input.teksti) {
      todos.push({ id: Date.now().toString(36) + Math.random().toString(36) });
      resultText = 'U shtua detyra.';
    } else if (tu.input.aksioni === 'kryer' && needle) {
      const t = todos.find(t => t.text.toLowerCase().includes(needle));
      resultText = t ? ((t.done = true), 'U shënua si e kryer.') : 'S\'u gj';
    } else if (tu.input.aksioni === 'fshij' && needle) {
      const before = todos.length;
      todos = todos.filter(t => !t.text.toLowerCase().includes(needle));
      resultText = todos.length < before ? 'U fshi detyra.' : 'S\'u gjet aj';
    } else {
      resultText = 'Veprim i paplotë – mungon teksti i detyrës.';
    }
    writeJSON(file, todos);
  }
  toolResults.push({ type: 'tool_result', tool_use_id: tu.id, content: resultText });
}
working.push({ role: 'user', content: toolResults });
}

return res.status(500).json({ error: true, message: 'Modeli përdori shumë mje'

} catch (err) {
  console.error('Gabim nga Anthropic API:', err);
  const status = err.status || 500;
  const detail = (err.error && err.error.error && err.error.error.message) || err.message;
  return res.status(status).json({ error: true, message: `[HTTP ${status}] ${detail}` });
});

```

```
app.listen(PORT, () => {  
  console.log(`SHOKU po punon → http://localhost:${PORT}`);  
});
```